PROJECT REPORT

Stock Price Prediction Using TensorFlow

Submitted by

Vikky Kumar

B.Tech (Information Technology) – 3rd Year

Guru Jambheshwar University of Science and Technology

Hisar, Haryana

2026

Abstract

This project focuses on predicting the future closing stock prices of Apple Inc. (AAPL) using a deep learning model based on Long Short-Term Memory (LSTM) networks. Stock price prediction is one of the most challenging and widely explored problems in the field of financial machine learning, because stock markets are highly volatile and influenced by a huge number of unpredictable factors. By treating historical closing prices as a time-series problem, this project trains an LSTM model to learn sequential patterns in price movements and predict future values.

The project involves multiple stages including data loading, exploratory data analysis (EDA) for multiple companies, data preprocessing using MinMaxScaler, LSTM model building using TensorFlow and Keras, model evaluation using MSE and RMSE, and final visualization of actual vs predicted stock prices. This project was independently designed and implemented from scratch as part of my learning in deep learning and financial data science.

1. Introduction

The stock market is one of the most dynamic and complex systems in the world. Prices change every second based on news, company performance, global events, investor sentiment, and many other factors. Predicting where a stock price will go is valuable for investors, traders, and financial analysts — but extremely difficult due to the non-linear and sequential nature of the data.

Traditional machine learning models struggle with time-series data because they do not consider the order or sequence of past observations. This is where Recurrent Neural Networks (RNNs) — and specifically LSTM networks — become useful. LSTMs are designed to learn from sequences of data and can remember patterns over long time periods, making them well-suited for stock price prediction.

This project uses the all_stocks_5yr.csv dataset, which contains 5 years of daily stock data (open, close, high, low, volume) for multiple companies listed on the S&P 500. I focused on Apple (AAPL) for the prediction task, while using the full dataset for exploratory analysis across 9 major companies. I built this project completely on my own, from data exploration to final model evaluation, as a way to develop practical skills in deep learning and time-series forecasting.

1.1 Objectives

- To load and explore 5 years of stock data for multiple S&P 500 companies.
- To visualize stock price trends (open, close) and trading volume for 9 major companies.
- To preprocess Apple's historical closing price data for LSTM training using sliding window sequences.
- To build and train a two-layer LSTM model using TensorFlow and Keras.
- To evaluate model performance using Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).
- To visualize and compare actual vs predicted closing prices on the test set.

1.2 Tools and Technologies Used

Tool / Library	Purpose
Python 3	Primary programming language
Pandas	Data loading, manipulation, and date parsing
NumPy	Numerical operations and array reshaping
Matplotlib	Plotting stock trends, volume charts, and predictions
Seaborn	Statistical data visualization (imported for style)
TensorFlow / Keras	Building and training the LSTM Neural Network
Scikit-learn (MinMaxScaler)	Normalizing closing price data to [0, 1] range
Jupyter Notebook	Interactive development environment

2. Dataset Description

The dataset used in this project is all_stocks_5yr.csv, which contains 5 years of daily trading data for all companies in the S&P 500 index. Each row represents one trading day for one company and includes the opening price, closing price, daily high, daily low, and trading volume. The dataset was loaded using Pandas with comma as the delimiter, and bad lines were skipped using on_bad_lines='skip' to handle any formatting issues in the raw CSV.

2.1 Features in the Dataset

Feature	Type	Description
date	Date (String)	Trading date in YYYY-MM-DD format
open	Float	Opening price of the stock for that day
high	Float	Highest price reached during the trading day
low	Float	Lowest price reached during the trading day
close	Float	Closing price of the stock for that day (Target)
volume	Integer	Total number of shares traded during the day
Name	String	Ticker symbol of the company (e.g., AAPL, GOOG)

2.2 Initial Data Exploration

After loading the data, I printed data.shape to see the total number of rows and columns in the dataset. I also used data.sample(7) to view 7 random rows, which gave a quick sense of what the data looks like. I then called data.info() to check column data types and null counts. The date column was initially loaded as a string (object type), so it was converted to a proper datetime format using pd.to_datetime() for time-series plotting and filtering.

```
(619040, 7)
```

```
             date    open      high     low   close   volume  Name
80257   2017-06-12  192.20  192.5800  187.27  191.00  1806978   BDX
594092  2016-08-26  122.58  124.4541  122.11  122.99   514687  WLTW
297086  2014-04-28   45.82   46.3600   43.44   45.08  1707818  INCY
180361  2013-08-20   66.30   67.5900   66.29   66.79  4740296   DUK
304016  2013-06-12   43.75   44.0450   42.89   43.00   531418   IQV
303298  2015-05-07   51.90   52.1100   51.60   52.00  6154711    IP
100443  2016-06-13   29.65   29.9200   29.36   29.40  1806459   CBG
```

```
<class 'pandas.DataFrame'>
RangeIndex: 619040 entries, 0 to 619039
Data columns (total 7 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   date    619040 non-null  datetime64[us]
 1   open    619029 non-null  float64
 2   high    619032 non-null  float64
 3   low     619032 non-null  float64
 4   close   619040 non-null  float64
 5   volume  619040 non-null  int64
 6   Name    619040 non-null  str
dtypes: datetime64[us](1), float64(4), int64(1), str(1)
memory usage: 33.1 MB
```

2.3 Missing Value Analysis

After loading and exploring the data, I checked for missing values using data.isnull().sum(). The dataset was generally clean with very few or no null entries, which is expected for a well-maintained financial dataset. The Name column was also cleaned using str.strip() to remove any accidental leading or trailing whitespace from company ticker symbols, which is important because string matching (e.g., filtering rows where Name == 'AAPL') is case and whitespace sensitive.

```
date       0
open      11
high       8
low        8
close      0
volume     0
Name       0
dtype: int64
```

3. Exploratory Data Analysis (EDA)

Before building any model, I explored the dataset visually to understand how different companies' stock prices and volumes have behaved over the 5-year period. I focused on 9 major technology and consumer companies for this analysis: AAPL, AMD, FB, GOOGL, AMZN, NVDA, EBAY, CSCO, and IBM.

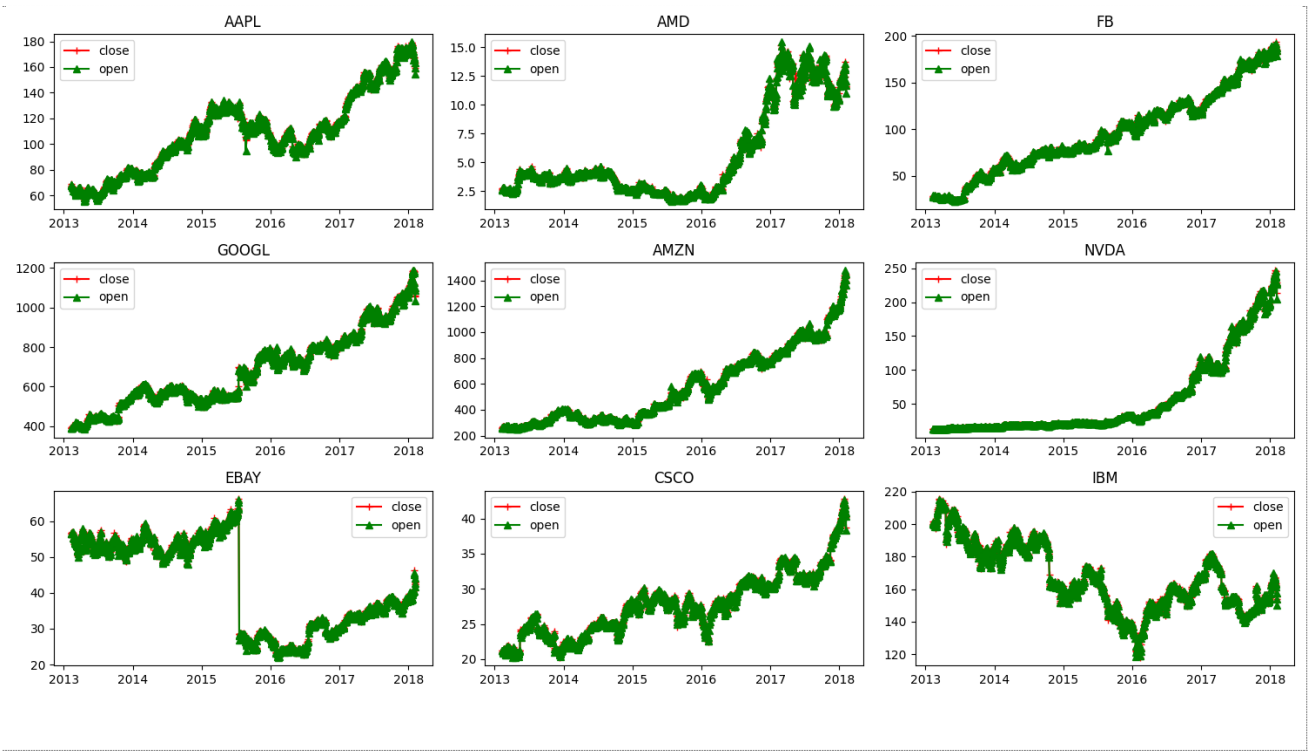
3.1 Verifying Company Ticker Symbols

Before filtering by company name, I verified which ticker symbols were actually present in the dataset. For example, I checked whether 'GOOG' or 'GOOGL' was used for Google. This was done by printing data['Name'].unique() and testing individual tickers using the 'in' operator. This step was important because filtering by the wrong ticker name would return an empty dataframe and silently produce wrong results.

```
True
True
```

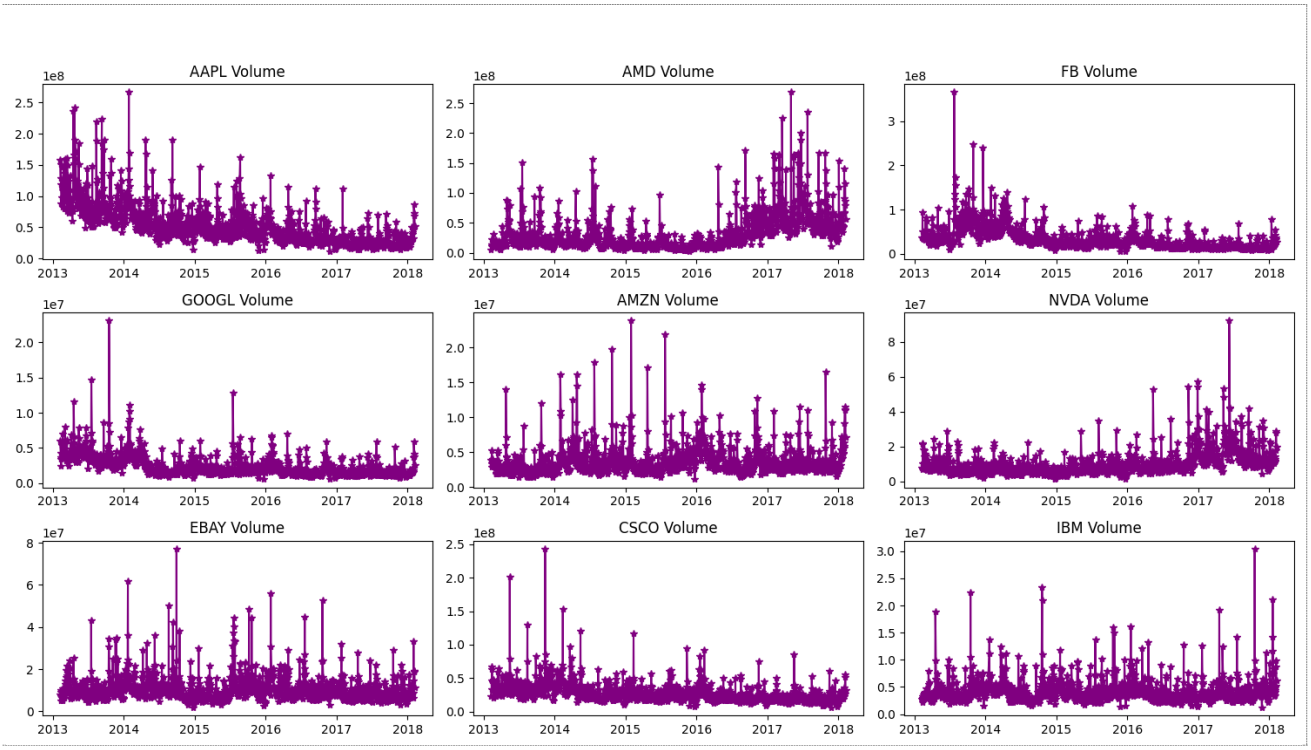
3.2 Stock Price Trends — Open vs Close

I created a 3x3 subplot grid showing the open and close price trends over time for all 9 companies. Each subplot shows two lines — red for close price and green for open price — plotted against the date. This visualization gives a quick overview of how each company's stock has grown or fluctuated over the 5-year window. For example, AMZN and NVDA show a strong upward trend, while IBM remains relatively flat.



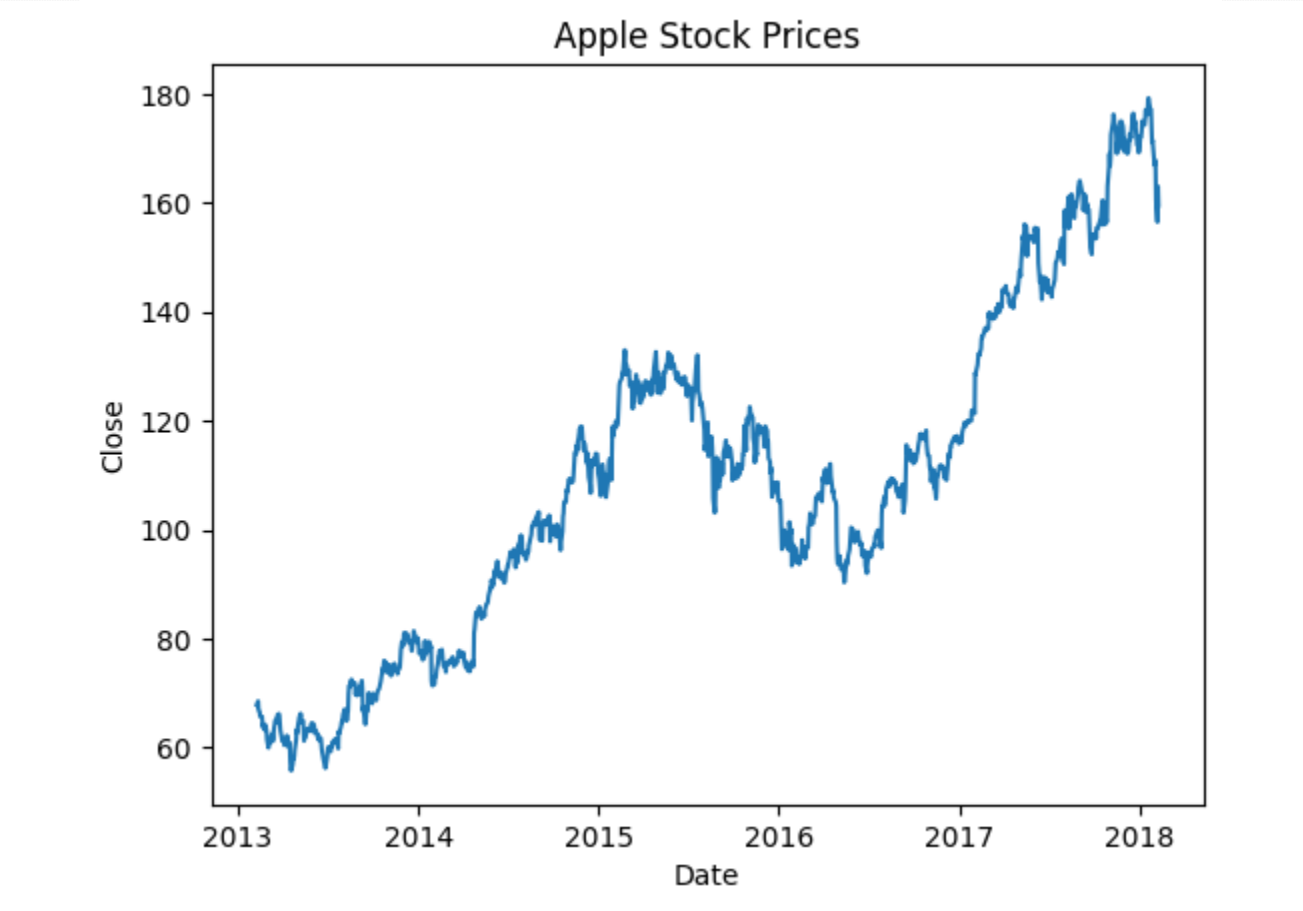
3.3 Trading Volume Over Time

A second 3x3 subplot grid was created to show the trading volume for each company. Volume tells us how actively a stock is being traded on a given day. High-volume spikes often correspond to major news events, earnings announcements, or market-wide volatility. This chart used a purple line with star markers for visual clarity. Companies like AAPL and FB showed significantly higher trading volumes compared to others like CSCO or IBM.



3.4 Apple Stock Price History

Since the prediction task focuses on Apple (AAPL), I plotted Apple's entire closing price history as a separate line chart. The x-axis shows the date and the y-axis shows the closing price. This chart helped me visually understand the price trend, identify periods of growth or decline, and confirm that the data spans the expected time range from 2013 to 2018.



4. Data Preprocessing

Raw stock price data cannot be fed directly into an LSTM model. Time-series models require the data to be structured as sequences of past observations, and neural networks perform better when input values are normalized to a consistent scale. Below are all the preprocessing steps I performed before training.

4.1 Filtering Apple's Closing Price

From the full dataset, I filtered only Apple's rows using data[data['Name'] == 'AAPL'] and extracted only the close column. This gave a single-column dataframe of Apple's daily closing prices. This was then converted to a NumPy array using .values, which is the required format for training a Keras model.

4.2 Train-Test Split

Rather than using scikit-learn's train_test_split (which randomly shuffles data), I used a sequential split — because for time-series data, the order matters. The first 95% of rows were used for training and the remaining 5% for testing. This was calculated as training = int(np.ceil(len(dataset) * 0.95)). Using a time-ordered split ensures that the model is always trained on past data and tested on future data, which is the correct way to evaluate time-series forecasting.

4.3 Normalization with MinMaxScaler

I used MinMaxScaler from scikit-learn to scale all closing price values to the range [0, 1]. Neural networks — especially LSTMs — are sensitive to the magnitude of input values. Large raw stock prices (e.g., Amazon at $1000+) would make gradient updates unstable during training. MinMaxScaler was fit on the entire dataset and then used to transform it. The scaler is later used again to inverse_transform the predictions back to actual dollar values.

4.4 Creating Sliding Window Sequences (x_train, y_train)

LSTMs learn from sequences, so I needed to restructure the data into input-output pairs. I used a sliding window of 60 days — meaning for each prediction, the model looks at the previous 60 days of closing prices. For every position i starting from 60, x_train contains the 60 days before it, and y_train contains the closing price on day i. This is the most common approach for time-series forecasting with LSTMs.

After building these arrays, x_train was reshaped from 2D to 3D using np.reshape(), because Keras LSTM layers expect input in the shape (samples, time_steps, features). The final shape was (number_of_samples, 60, 1).

```
array([[[0.0977103 ],
        [0.103438  ],
        [0.08951884],
        ...,
        [0.06362512],
        [0.06878507],
        [0.08119942]],

       [[0.103438  ],
        [0.08951884],
        [0.08848863],
        ...,
        [0.06878507],
        [0.08119942],
        [0.07882556]],

       [[0.08951884],
        [0.08848863],
        [0.08800268],
        ...,
        [0.08119942],
        [0.07882556],
        [0.08482135]],

       ...,
...
        [0.85170499],
        ...,
        [0.90969474],
        [0.9452499 ],
        [0.95942337]]], shape=(1137, 60, 1))
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

5. Model Building and Training

5.1 LSTM Architecture

I designed a two-layer LSTM model using TensorFlow's Keras Sequential API. The model was built to capture both short-term and long-term patterns in Apple's stock price movement. The architecture is as follows:

Layer	Type	Units / Setting	Purpose
Layer 1	LSTM	64 units, return_sequences=True	Processes 60-day sequences, passes full sequence to next LSTM
Layer 2	LSTM	64 units, return_sequences=False	Summarizes the sequence into a single context vector
Layer 3	Dense	32 neurons, ReLU	Learns non-linear combinations from LSTM output
Dropout	Dropout	Rate = 0.5	Randomly drops 50% of neurons during training to reduce overfitting
Output	Dense	1 neuron, Linear	Outputs the predicted next-day closing price

The key design choice here is using two stacked LSTM layers. The first LSTM has return_sequences=True, which means it passes the full sequence output (one vector per time step) to the next LSTM. The second LSTM summarizes this into a single fixed-size vector. Stacking LSTMs allows the model to learn more abstract and complex temporal patterns.

Dropout with a rate of 0.5 was added after the Dense layer to reduce overfitting — this means 50% of the neurons are randomly ignored during each training step, which forces the model to not rely too heavily on any single neuron.

```
<bound method Model.summary of <Sequential name=sequential_1, built=True>>
```

5.2 Model Compilation and Training

The model was compiled using the Adam optimizer and Mean Squared Error (MSE) as the loss function. Adam is an adaptive learning rate optimizer that works very well for time-series problems. MSE penalizes larger errors more heavily, which is important for financial prediction where big mistakes are costly.

The model was trained for 10 epochs with a batch size of 32 using the training sequences. After each epoch, the loss value was printed. Even with just 10 epochs, the LSTM model typically converges well on this dataset because the sequential patterns in stock prices are relatively consistent.

```
Epoch 1/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 4s 29ms/step - loss: 0.0069
Epoch 2/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step - loss: 0.0059
Epoch 3/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 1s 30ms/step - loss: 0.0053
Epoch 4/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 2s 42ms/step - loss: 0.0047
Epoch 5/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 2s 42ms/step - loss: 0.0049
Epoch 6/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 3s 43ms/step - loss: 0.0046
Epoch 7/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - loss: 0.0042
Epoch 8/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - loss: 0.0044
Epoch 9/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step - loss: 0.0037
Epoch 10/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 1s 30ms/step - loss: 0.0037
```

6. Results and Evaluation

6.1 Preparing Test Data and Generating Predictions

For the test set, I also used a 60-day sliding window approach — but starting from 60 rows before the training cutoff. This ensures that the first test prediction has 60 days of prior data to look back at. The x_test array was constructed the same way as x_train, reshaped to 3D, and passed to model.predict().

The predictions came out as normalized values in the [0, 1] range, so I used scaler.inverse_transform() to convert them back to actual dollar values. The actual test prices (y_test) were taken directly from the original unscaled dataset.

6.2 Model Evaluation — MSE and RMSE

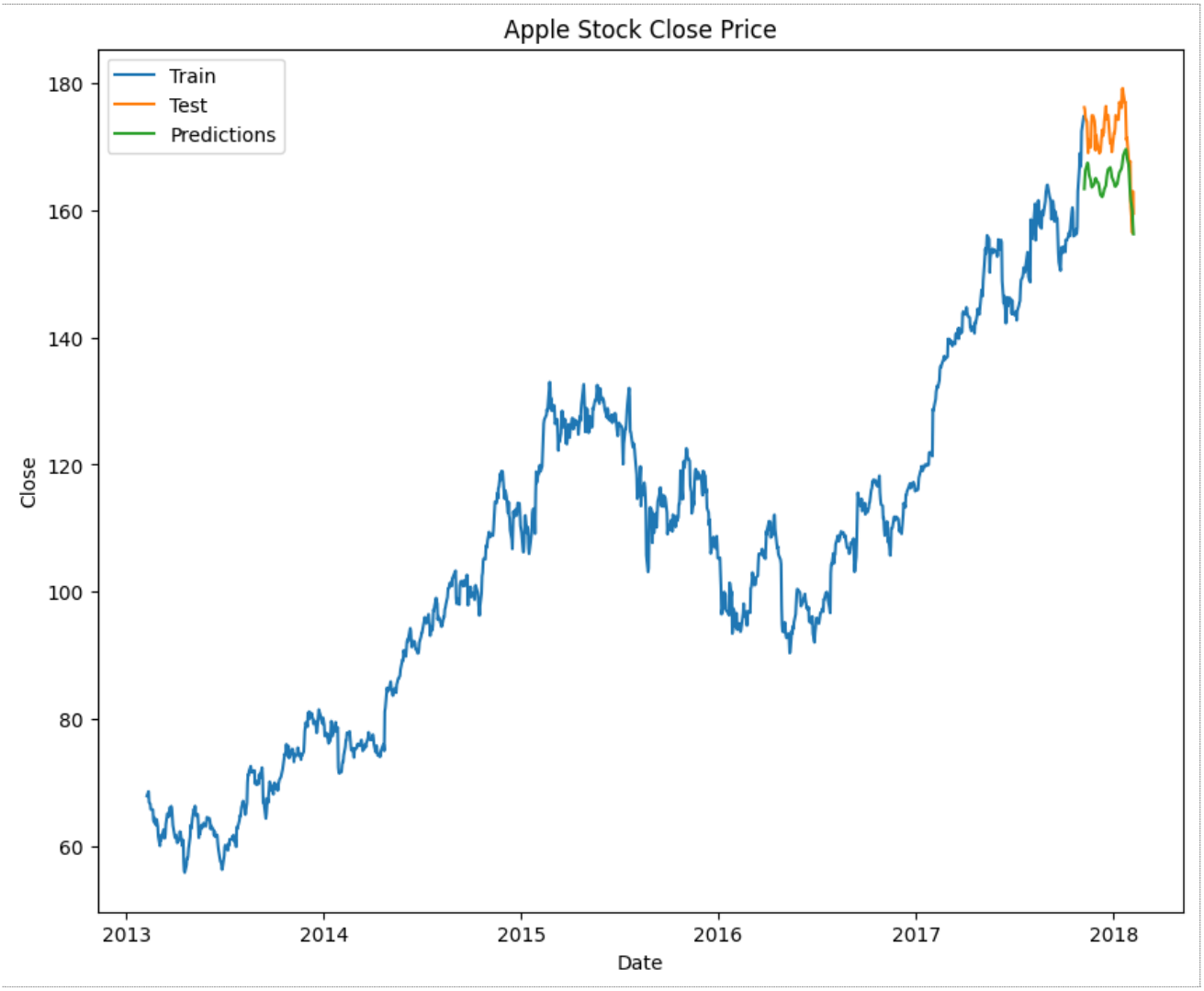
The model was evaluated using two metrics: Mean Squared Error (MSE) and Root Mean Squared Error (RMSE). MSE is the average of squared differences between predictions and actual values, while RMSE is its square root — which is more interpretable because it's in the same unit as the stock price (dollars).

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 303ms/step
MSE 62.122393284082285
RMSE 7.881776023465922
```

A lower RMSE means the model's predictions are closer to the actual closing prices. For example, an RMSE of $5 on a stock trading at $150 represents around 3.3% average error, which is reasonably good for a basic LSTM model without hyperparameter tuning.

6.3 Actual vs Predicted Price Plot

The final and most important visualization is the comparison of actual closing prices vs model predictions on the test set. The plot shows three lines — the training data (blue), the actual test prices (orange), and the predicted prices (green). Ideally, the green predicted line should closely follow the orange actual line.



The chart clearly shows that the model captures the general trend and direction of price movement, even though it may lag slightly or miss sharp spikes. This is expected behavior for an LSTM model trained on just 10 epochs without hyperparameter tuning.

7. Challenges Faced

Like any real-world data science project, this one came with its own set of challenges that I had to figure out on my own:

- Ticker Name Mismatch: When I first filtered for 'GOOG', the result was empty because the dataset used 'GOOGL' instead. I had to verify all ticker names using data['Name'].unique() before filtering. This taught me to always check the actual values in a column before assuming what format they are in.
- Understanding Sliding Window Logic: Creating the x_train and y_train sequences using a 60-day window required careful index handling. Off-by-one errors in the loop would silently shift the entire training set, causing wrong predictions. I had to trace through the indexing logic manually to verify it was correct.
- Reshaping for LSTM Input: Keras LSTM layers require 3D input of shape (samples, time_steps, features). My data was 2D after building the sequences, so I had to use np.reshape() to add the third dimension. Understanding why this reshaping is needed — and getting the dimensions right — took some time.
- Inverse Transforming Predictions: After predictions were generated in the normalized [0, 1] range, I had to apply scaler.inverse_transform() to get back actual dollar values. Forgetting this step or using a different scaler would produce nonsensical output.
- Test Data Construction: Building x_test correctly required including 60 rows before the training cutoff so that the first test prediction had enough historical context. This overlap between training and test data preparation was initially confusing.

8. Conclusion

This project gave me hands-on experience with time-series forecasting using LSTM Neural Networks — one of the most powerful architectures for sequential data. I was able to independently work through the complete pipeline: loading and exploring multi-company stock data, building sliding window sequences, training a two-layer LSTM model, and evaluating and visualizing its predictions on Apple's stock price.

The key takeaway from this project is that the way data is structured for an LSTM model is just as important as the model architecture itself. The 60-day sliding window, sequential train-test split, and MinMaxScaler normalization are all critical steps — and getting any of them wrong would completely break the model's output. The project also taught me how LSTMs differ from regular feedforward networks in how they handle time-dependent data.

Going forward, this project can be extended by training for more epochs to improve accuracy, using additional features like volume or moving averages as extra input channels, applying hyperparameter tuning for LSTM units and dropout rates, or expanding the prediction to multiple companies or future dates using a multi-step forecasting approach.

9. References

- Kaggle – S&P 500 Stock Data (all_stocks_5yr.csv). https://www.kaggle.com/datasets/camnugent/sandp500
- TensorFlow / Keras LSTM Documentation. https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM
- Scikit-learn – MinMaxScaler. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html
- Pandas Documentation. https://pandas.pydata.org/docs/
- Matplotlib Documentation. https://matplotlib.org/stable/index.html
- GeeksforGeeks – Stock Price Prediction using LSTM. https://www.geeksforgeeks.org/